

Greedy Algorithms

CMSC 27200: Theory of Algorithms

January 10-15, 2024

Corresponding section(s) of Kleinberg-Tardos: 4.1, 4.2, 4.4, 4.5

1 Overview

Greedy algorithms maximize the immediate payoff according to some rule (which depends on the problem and the algorithm). Greedy algorithms are not always optimal, as it may be better to take a smaller gain now in order to obtain a larger gain later. That said, greedy algorithms have the advantage that they are fast and relatively simple and there are many problems where there is a greedy algorithm which is optimal or at least gives a good approximation.

2 Variants of Interval Scheduling

Problem 2.1 (Interval Scheduling: Maximizing the Number of Jobs Accepted). *In this interval scheduling problem, we have one processor and we are given n jobs. Each job has a set time interval $[a_i, b_i]$ when it must use the processor if it is accepted. We are then asked to find a schedule S which accepts the maximum number of jobs without having two jobs which run on the processor at the same time.*

This problem can be stated more precisely using the following definition:

Definition 2.2. *Given n intervals $[a_1, b_1], \dots, [a_n, b_n]$, we say that a sequence $S = (i_1, \dots, i_m)$ of elements of $[n]$ is a valid schedule if $\forall j \in [m-1] (b_{i_j} \leq a_{i_{j+1}})$ (the jobs $i_1, \dots, i_m$ are in sequential order and job i_{j+1} does not start before job i_j finishes). We define $|S| = m$ to be the number of elements in S (i.e. the number of jobs accepted).*

Precise problem statement: Given n intervals $[a_1, b_1], \dots, [a_n, b_n]$, find a valid schedule $S = (i_1, \dots, i_m)$ which maximizes $|S| = m$.

Algorithm 2.3. *This problem can be solved with the following greedy algorithm: When choosing the next job, **always choose the job which can be finished first**. More precisely,*

Preprocessing: We sort the jobs in order of their end time b_i . For simplicity, we relabel the jobs so that $b_1 \leq b_2 \leq \dots \leq b_n$.

Stored data: We keep track of a valid schedule $S_k = (i_1, \dots, i_k)$ of the jobs which we have accepted so far and the current job j we are considering.

Initialization: We start with $S_1 = (1)$, $k = 1$, and $j = 2$ as we always want to take $i_1 = 1$ (with the relabeling, job 1 finishes first).

Iterative step: Given the current schedule $S_k = (i_1, \dots, i_k)$, if $a_j \geq b_{i_k}$ then we add job j to our schedule by setting $i_{k+1} = j$ and incrementing j and k . If $a_j < b_{i_k}$ then we discard job j and increment j .

We stop when $j = n + 1$.

Theorem 2.4. *If the greedy algorithm gives a valid schedule $S = (i_1, \dots, i_m)$ then for any other valid schedule $S' = (i'_1, \dots, i'_{m'})$, $m' \leq m$.*

Proof. The intuition is that by always choosing the job which finishes first, S completes jobs as fast as possible and thus it is impossible to get ahead of S . We can turn this intuition into a proof using the following lemma, which says that the greedy algorithm always stays ahead:

Lemma 2.5. *If $S = (i_1, \dots, i_m)$ is the schedule given by the greedy algorithm then for any other valid schedule $S' = (i'_1, \dots, i'_{m'})$, for all $j \in [m]$, either $m' < j$ or $b_{i_j} \leq b_{i'_j}$ (which is equivalent to $i_j \leq i'_j$ since $b_1 \leq b_2 \leq \dots \leq b_n$).*

Proof. We prove this lemma by induction. The base case $j = 1$ is trivial as $i_1 = 1$ so $b_1 \leq b_{i'_1}$ (unless $m' = 0$ in which case $m' < j$). For the inductive step, assume that $b_{i_k} \leq b_{i'_k}$ for some $k < m$. If $m' < k + 1$ then we are done. Otherwise, note that $a_{i'_{k+1}} \geq b_{i'_k} \geq b_{i_k}$ so job i'_{k+1} is available to S . Since S always **chooses the job with the earliest completion time** out of the available jobs, $b_{i_{k+1}} \leq b_{i'_{k+1}}$, as needed. $\square$

Applying this lemma with $j = m$, either $m' < m$ or $b_{i_m} \leq b_{i'_m}$. If $b_{i_m} \leq b_{i'_m}$ then since there are no jobs starting after b_{i_m} (as otherwise the greedy algorithm would have taken such a job), S' cannot have any more jobs and thus $m' = m$. $\square$

The runtime of this algorithm is $O(n \log n)$ as the initial sorting takes time $O(n \log n)$ and the remaining steps take $O(n)$ as each iterative step takes $O(1)$ time.

2.1 A different interval scheduling problem

Problem 2.6 (Interval Scheduling: Minimizing Maximum Lateness). *In this interval scheduling problem, we are given n jobs, each of which has a length $l_i > 0$ and a deadline d_i (and can be scheduled at any time $t \geq 0$). We are then asked to complete the jobs one by one so that the maximum lateness of any single job is minimized.*

Example 2.7. *If we have two jobs with lengths and deadlines $l_1 = 4, d_1 = 3, l_2 = 6, d_2 = 2$ then we have the following options:*

1. *If we schedule job 1 before job 2 then job 1 will finish at time 4 and will be late by 1 and job 2 will finish at time 10 and will be late by 8.*

2. If we schedule job 2 before job 1 then job 2 will finish at time 6 and will be late by 4 and job 1 will finish at time 10 and will be late by 7.

To minimize the maximum lateness, it is better to schedule job 2 first as $\max\{4, 7\} = 7 < \max\{1, 8\} = 8$. Note that this does not minimize the total lateness.

This problem can be stated more precisely using the following definition:

Definition 2.8. Given a schedule $S = (i_1, \dots, i_n)$ where $(i_1, \dots, i_n)$ is a permutation of $[n]$, define $t_S(i_k) = \sum_{j=1}^k l_{i_j}$ to be the time at which job i_k is finished

With this definition, our problem can be restated as follows. Find a schedule $S = (i_1, \dots, i_n)$ where $(i_1, \dots, i_n)$ is a permutation of $[n]$ such that $\max_{j \in [n]} \{t_S(i_j) - d_{i_j}\}$ is minimized.

Algorithm 2.9 (Earliest deadline first). Take the schedule $S_{\text{greedy}} = (i_1, \dots, i_n)$ such that $d_{i_1}, \dots, d_{i_n}$ are in increasing order.

Theorem 2.10. The schedule $S = (i_1, \dots, i_n)$ such that $d_{i_1}, \dots, d_{i_n}$ are in increasing order minimizes $\max_{j \in [n]} \{t_S(i_j) - d_{i_j}\}$.

Proof. The idea behind the proof is as follows. Given an optimal schedule S which is not equal to S_{greedy} , we can transform S into S_{greedy} by exchanging pairs of jobs without increasing $\max_{j \in [n]} \{t_S(i_j) - d_{i_j}\}$. This shows that S_{greedy} is optimal.

Lemma 2.11. If $S = (i_1, \dots, i_n)$ is an optimal schedule, $j \in [n - 1]$, and $d_{i_j} > d_{i_{j+1}}$ then the schedule S' obtained by **swapping i_j and i_{j+1}** is also optimal.

Proof. We make the following observations:

1. Swapping i_j and i_{j+1} does not affect the lateness of any other job as for all $j' \in [n] \setminus \{j, j+1\}$, $t_{S'}(i_{j'}) = t_S(i_{j'})$.
2. $t_S(i_{j+1}) = t_{S'}(i_j) = t_{S'}(i_{j+1}) + l_{i_j}$ as schedule S' puts job i_{j+1} before job i_j .

This implies that $t_S(i_{j+1}) - d_{i_{j+1}} > t_{S'}(i_{j+1}) - d_{i_{j+1}}$ and $t_S(i_j) - d_{i_j} > t_{S'}(i_j) - d_{i_j}$ as $d_{i_j} > d_{i_{j+1}}$. Thus, the lateness of job i_{j+1} for schedule S is greater than the lateness of both job i_j and i_{j+1} for schedule S' .

Putting these observations together, the maximum lateness for S' is less than or equal to the maximum lateness for S . $\square$

Using Lemma 2.11, whenever two adjacent jobs are out of order, we can swap them and put them in order. If we keep on doing this, the schedule will get closer to being in order with each step and this process can only terminate when everything is in order at which point we have reached the schedule S given by the greedy algorithm.

To make this argument rigorous, we use the lexicographical ordering on the deadlines.

Definition 2.12. Given two schedules $S = (i_1, \dots, i_n)$ and $S' = (i'_1, \dots, i'_n)$, we say that $S < S'$ if there is a $k \in [n]$ such that $d_{i_k} < d_{i'_k}$ and for all $j \in [k - 1]$, $i_j = i'_j$.

Let $S' = (i'_1, \dots, i'_n)$ be the optimal schedule which comes first in this lexicographical ordering, i.e., for any other optimal schedule S'' , $S' < S''$. We claim that $S' = S$. To see this, assume $S' \neq S$. If so, there exists a $j \in [n - 1]$ such that $d_{i'_j} > d_{i'_{j+1}}$. But then by Lemma 2.11, the schedule S'' obtained by swapping i'_j and i'_{j+1} in S' is also optimal and we have that $S'' < S'$, which contradicts how we chose S' . $\square$

3 Review: Depth First Search and Breadth First Search

Problem 3.1 (Directed Connectivity). *In the directed connectivity problem, we are given a directed graph $G = (V, E)$ and we are asked whether there is a path from some vertex s to another vertex t .*

Directed connectivity can be solved with the following kind of algorithm.

Algorithm 3.2 (Algorithms for directed connectivity).

Stored data: We keep track of a set $S \subseteq V$ of vertices which we know are reachable from s . We also keep track of a set $E_{unexplored}$ of edges where for each $(u, v) \in E_{unexplored}$ we have reached u but may or may not have reached v .

Initialization: We start with $S = \{s\}$ and $E_{unexplored} = \{(s, v) : (s, v) \in E\}$.

Iterative step: At each step we remove an edge $(u, v) \in E_{unexplored}$ if possible and do the following:

1. If this is not possible because $E_{unexplored}$ is empty then we output NO because we have already reached everything which is reachable from s and we have not reached t .
2. If $v \in S$ then we do nothing as v has already been reached before.
3. If $v \neq t$ and $v \notin S$ then we add v to S and add all of the edges $\{(v, w) : (v, w) \in E\}$ to $E_{unexplored}$.
4. If $v = t$ then we stop and output YES because we have reached our destination.

This kind of algorithm solves directed connectivity and takes $O(|E|)$ time because each edge of G is added and removed from $E_{unexplored}$ at most once. However, to fully describe such an algorithm we need to describe how we add and remove edges from $E_{unexplored}$. The two standard choices for this are as follows:

1. If $E_{unexplored}$ is stored as a stack then this is the depth first search algorithm.
2. If $E_{unexplored}$ is stored as a queue then this is the breadth first search algorithm.

4 Dijkstra's Algorithm for Shortest Path

Problem 4.1 (Shortest Path Problem). *In the shortest path problem, we are given the edge lengths of a directed graph G on n vertices (which must be non-negative). We are then asked to find a path of minimum length from some vertex s to another vertex t .*

More precisely, we are given the following data:

1. A directed graph $G = (V, E)$ together with a starting vertex $s \in V$ and a destination vertex $t \in V$.
2. For each edge $e = (u, v) \in E$, we are given its length $l_e = l_{uv}$ which must be non-negative.

We are then asked to output a path P from s to t such that $\sum_{e \in E(P)} l_e$ is minimized.

Algorithm 4.2 (Dijkstra's Algorithm). *Dijkstra's algorithm finds a path of minimum length from s to t by iteratively finding the paths of minimum length from s to every other vertex in G . To do this, Dijkstra's algorithm keeps track of candidate paths to new locations and then takes the shortest candidate path, which is guaranteed to be correct.*

More precisely, Dijkstra's algorithm can be described as follows.

Stored data: We keep track of the following:

1. *A set $S \subseteq V$ together with distances $\{d_v : v \in S\}$ where for all $v \in S$, d_v is the length of the shortest path P_v from s to v . For each $v \in S \setminus \{s\}$, we also keep track of the last edge (u, v) of P_v . Working backwards, this is sufficient to reconstruct P_v .*
2. *A set of edges $E_{\text{candidate}}$ together with distances $\{d_e : e \in E_{\text{candidate}}\}$ where for all $e = (v, w) \in E_{\text{candidate}}$, $d_e = d_v + l_e$ is the length of the path formed by taking the shortest path from s to v and adding the edge e .*

Initialization: We start with $S = \{s\}$, $d_s = 0$, $E_{\text{candidate}} = \{(s, v) : (s, v) \in E\}$, and $d_e = l_e$ for every edge $e = (s, v) \in E_{\text{candidate}}$

Iterative step: At each step we remove the edge $e = (v, w) \in E_{\text{candidate}}$ which has the smallest d_e and do the following:

1. *If this is not possible because $E_{\text{candidate}}$ is empty then we output that t is unreachable from s because we have already found the shortest paths to every vertex which is reachable from s and we have not reached t .*
2. *If $w \in S$ then we do nothing as we have already found a path of equal or smaller length to w .*
3. *If $w \neq t$ and $w \notin S$ then we add w to S , take $d_w = d_e = d_v + l_{vw}$, note that (v, w) was the last edge used to reach w , and add all of the edges $\{(w, x) : (w, x) \in E\}$ to $E_{\text{candidate}}$ with distances $d_{e'} = d_w + l_{e'}$.*
4. *If $w = t$ then we add t to S , take $d_t = d_e = d_v + l_{vt}$, and note that (v, t) was the last edge used to reach t . We then stop and output d_t as we have found the shortest path from s to t .*

Theorem 4.3. *Dijkstra's algorithm finds a path from s to t of minimum length and can be implemented in $O(|E|\log(|E|))$ time.*

Proof. To prove that Dijkstra's algorithm finds the minimum length of a path from s to t , we prove the stronger statement that for all $v \in S$, d_v is the minimum length of a path from s to v .

Lemma 4.4. *We always have that for all $v \in S$, d_v is the minimum length of a path from s to v .*

Proof. We prove this by induction. The base case when $S = \{s\}$ is trivial as the shortest path from s to s has length 0. For the inductive step, assume that this statement holds when $|S| = k$ and consider what happens when we add the next vertex w to S via an edge (v, w) where $v \in S$. We claim that $d_w = d_v + l_{vw}$ is the minimum length of a path from s to w . To see this, assume that there is another path P' from s to w which has smaller length. Let w' be the first vertex on this path which is not in S and let (v', w') be the edge used to reach w' .

Observe that by the inductive hypothesis, the length of the part of P' between s and v' is at least $d_{v'}$. Moreover, we must have that $d_{v'} + l_{v'w'} \geq d_v + l_{vw} = d_w$ because otherwise Dijkstra's algorithm would have taken the edge (v', w') and the vertex w' rather than (v, w) and w . Thus, the length of the part of P' between s and w' is at least d_w , which contradicts the assumption that the length of P' is less than d_w .

Note that for this argument to work, it is important that the edge weights are non-negative as otherwise we could have that $d_{v'} + l_{v'w'} < d_v + l_{vw} = d_w$ even though $d_{v'} > d_v$. We will discuss how to handle negative weights later in the course. $\square$

To implement Dijkstra's algorithm, we can use a priority queue (see the lecture notes on data structures) to store the candidate edges (v, w) and distances d_e . Each edge e is added at most once and removed at most once, so we perform at most $2|E|$ operations on this priority queue. This priority queue never has more than $|E|$ elements, so each operation takes time $O(\log(|E|))$. Thus, the total time required is $O(|E|\log(|E|))$ $\square$

Remark 4.5. *Instead of storing the edges of G in the priority queue, we can instead store the unreached vertices of G in the priority queue (together with the length of the shortest path to the vertex we've seen so far and the last edge of this path). Whenever we encounter a shorter path to an unreached vertex w , we decrease the weight of w .*

Using this approach, Dijkstra's algorithm can actually be implemented in time $O(|E| + |V|\log(|V|))$ by using Fibonacci heaps (which can decrease the weight of an element in $O(1)$ time) to implement the priority queue. This is slightly faster in theory but is generally slower in practice because Fibonacci heaps are somewhat complicated.

5 Kruskal's Algorithm and Prim's Algorithm for finding Minimum Spanning Trees

Definition 5.1. *Given an undirected graph $G = (V, E)$, we say that $T \subseteq E$ is a spanning tree of G if (V, T) is a tree (i.e. (V, T) is connected and contains no cycles).*

Problem 5.2. *The minimum spanning tree problem is as follows:*

Input: An undirected graph $G = (V, E)$ together with a length l_e for every edge $e \in E$.

Output: A spanning tree T of G such that $\sum_{e \in T} l_e$ is minimized.

For simplicity, we assume that G is connected (as otherwise G does not have any spanning trees) and that all of the edges of G have distinct lengths. If G has multiple edges which have the same length, we can add a small perturbation to the edge lengths to make them all distinct and then return the resulting minimum spanning tree.

Algorithm 5.3 (Kruskal's Algorithm). *Kruskal's algorithm first sorts the edges E in increasing order of their length. Kruskal's algorithm then considers the edges one by one and accepts each edge which does not form a cycle with the edges which were previously accepted.*

More precisely, Kruskal's algorithm can be described as follows:

Stored data: We keep track of a set of edges T and the set $E_{\text{remaining}}$ of the remaining edges of G which we have not yet considered. $E_{\text{remaining}}$ will always be sorted by length.

Initialization: We start with $T = \{\}$ and $E_{\text{remaining}} = E$ (sorted by length).

Iterative step: We remove the first edge e from $E_{\text{remaining}}$ (which will be the shortest edge we have not yet considered). If $T \cup \{e\}$ contains a cycle then we discard e and continue. Otherwise, we accept e by adding e to T . We stop when T is a spanning tree (in which case T will be the minimum spanning tree) or when $E_{\text{remaining}}$ is empty (in which case the original graph was not connected and there is no spanning tree).

Algorithm 5.4 (Prim's Algorithm). *Prim's algorithm is similar to Dijkstra's algorithm. Prim's algorithm starts at a vertex s and then builds a tree out from s by iteratively accepting the shortest edge which leads to a new vertex.*

More precisely, Prim's algorithm can be described as follows:

Stored data: We keep track of the following:

1. The set of edges T in our current tree and the set of vertices S in our current tree.
2. A set of edges $E_{\text{candidate}}$ together with their lengths l_e .

Initialization: We start with $S = \{s\}$, $T = \{\}$, $E_{\text{candidate}} = \{\{s, v\} : \{s, v\} \in E\}$

Iterative step: At each step we remove the shortest edge $e = \{v, w\} \in E_{\text{candidate}}$ and do the following:

1. If this is not possible because $E_{\text{candidate}}$ is empty then we output that there is no spanning tree because G is not connected.
2. If $w \in S$ then we do nothing as w is already part of the tree T .
3. If $w \notin S$ then we add w to S , add $e = \{v, w\}$ to T , and add all of the edges $\{\{w, x\} : \{w, x\} \in E\}$ to $E_{\text{candidate}}$ with their lengths l_e . If $S = V$ then we stop and output T .

As we will see, both Kruskal's algorithm and Prim's algorithm give us the minimum spanning tree. The fundamental reason for this is that there is a nice characterization of the edges of the minimum spanning tree.

Remark 5.5. *Prim's algorithm can be implemented in the same way as Dijkstra's algorithm and thus takes $O(|E|\log(|E|))$ time. To implement Kruskal's algorithm, we first need to sort the edges by length, which takes $O(|E|\log(|E|))$ time. We also need a way to test whether $T \cup \{e\}$ contains a cycle. This can be done using the union-find data structure in $O(\alpha(n))$ amortized time per query (i.e. some queries may take longer but the time required for n queries is $O(n\alpha(n))$). Here $\alpha(n)$ is the inverse Ackermann function which is not quite constant but grows incredibly slowly. For all practical purposes, $\alpha(n) \leq 4$.*

Theorem 5.6. *If $G = (V, E)$ is a connected graph where each edge $e \in E$ has a length $l_e > 0$ and these edge lengths are all distinct then there is a unique minimum spanning tree T . Moreover, each edge $e = \{u, v\} \in E$ is in T if and only if there exists a cut $C = (L, R)$ (i.e., a partition of the vertices of G into two sets L and R) such that e is the shortest edge between L and R .*

Proof. Let T be a minimum spanning tree of G and let $e = \{u, v\}$ be an edge of G .

Assume that $e \notin T$ and there exists a cut $C = (L, R)$ such that e is the shortest edge between L and R . Since T is a tree, adding e to T must create a cycle and this cycle must contain another edge e' which goes between L and R . Now observe that $T' = (T \cup \{e\}) \setminus e'$ is also a spanning tree. Since e is the shortest edge between L and R , $l_e < l_{e'}$. This implies that $\sum_{e \in T'} l_e < \sum_{e \in T} l_e$ which contradicts the assumption that T is a minimum spanning tree.

Conversely, assume that $e \in T$. Let L be the set of vertices u' such that there is a path from u to u' in T which does not go through v . Let R be the set of vertices v' such that there is a path from v to v' in T which does not go through u .

We have that e is the shortest edge between L and R . To see this, assume that there is a shorter edge e' between L and R . If so, then $T' = (T \cup \{e'\}) \setminus \{e\}$ would be a spanning tree with shorter total edge length than T which contradicts the assumption that T is a minimum spanning tree. $\square$

Theorem 5.7. *Given a graph $G = (V, E)$ with edge lengths which are all distinct, Kruskal's algorithm and Prim's algorithm both give the minimum spanning tree T of G .*

Proof. To see that Kruskal's algorithm gives T , we make the following observations:

1. If Kruskal's algorithm adds an edge $e = \{u, v\}$, let L be the set of vertices u' such that there is a path from u to u' using the edges which have been added so far and let $R = V \setminus L$.

Note that $v \notin L$ as otherwise adding e would create a cycle. By the definition of L , no edges between L and R were added. Now observe that there cannot be any edge e' between L and R which is shorter than e as adding e' does not create a cycle so e' would have been added before encountering e . Thus, e must be the shortest edge between L and R so $e \in T$.

2. If there exists a cut $C = (L, R)$ such that e is the shortest edge between L and R then e is the first edge between L and R which Kruskal's algorithm will consider. At this point, no other edges between L and R have been added so adding e will not create a cycle and thus Kruskal's algorithm will take e .

To see that Prim's algorithm gives T , we make the following observations:

1. Whenever Prim's algorithm adds an edge e , e is the shortest edge between the set S of vertices which have been reached so far and $V \setminus S$ so $e \in T$.
2. If e is an edge of T then when Prim's algorithm considers e , it will add e . To see this, observe that since Prim's algorithm only adds edges in T , adding e cannot create a cycle so e must go to a new vertex.

$\square$

5.1 Alternative characterization of the edges of a minimum spanning tree

There is a nice alternative characterization of the edges of a minimum spanning tree which can also be used to show that Kruskal's algorithm and Prim's algorithm are correct.

Definition 5.8. *Given a graph $G = (V, E)$ with edge lengths which are all distinct, define $E_{\text{redundant}}$ to be the set of edges e such that G contains a cycle C such that e is the longest edge in C , i.e.*

1. $e \in E(C)$
2. $\forall e' \in E(C) \setminus \{e\} (l(e') < l(e))$

Theorem 5.9. *If G is connected and all of the lengths of the edges of G are distinct then there is a unique minimum spanning tree T of G with edges $E(T) = E(G) \setminus E_{\text{redundant}}$.*

To prove this theorem, we need the following lemma which says that it is sufficient to consider cycles where only one edge is in $E_{\text{redundant}}$.

Lemma 5.10. *Given a graph $G = (V, E)$ with edge lengths which are all distinct, if $e \in E_{\text{redundant}}$ then G contains a cycle C such that e is the longest edge in C and all other edges of C are not in $E_{\text{redundant}}$, i.e.*

1. $e \in E(C)$
2. $\forall e' \in E(C) \setminus \{e\} (l(e') < l(e) \text{ and } e' \notin E_{\text{redundant}})$

Proof. Assume this lemma is not true and let e be the shortest edge such that $e = \{u, v\} \in E_{\text{redundant}}$ and there is no cycle C in G such that

1. $e \in E(C)$
2. $\forall e' \in E(C) \setminus \{e\} (l(e') < l(e) \text{ and } e' \notin E_{\text{redundant}})$

If so, then take $E' = \{e' \in E : l(e') < l(e), e' \notin E_{\text{redundant}}\}$ and take $G' = (V, E')$. Observe that u and v are disconnected in G' as otherwise we would have such a cycle C . However, since $e \in E_{\text{redundant}}$, there is a cycle C' in G such that

1. $e \in E(C')$
2. $\forall e' \in E(C') \setminus \{e\} (l(e') < l(e))$

Let $w_1, \dots, w_m$ be the path from $w_1 = u$ to $w_m = v$ in C' which does not use the edge $\{u, v\}$ and let j be the minimum index such that w_1 is not connected to w_j in G' (such a j must exist because w_1 and w_m are disconnected in G'). w_1 is connected to w_{j-1} in G' so w_{j-1} must be disconnected from w_j in G' and thus $(w_{j-1}, w_j) \notin E(G')$. Since $l_{(w_{j-1}, w_j)} < l_e$, this implies that $e' = (w_{j-1}, w_j) \in E_{\text{redundant}}$.

Moreover, since w_{j-1} must be disconnected from w_j in G' there is no cycle C'' such that

1. $e' = \{w_{j-1}, w_j\} \in E(C'')$
2. $\forall e'' \in E(C'') \setminus \{e'\}, l(e'') < l(e') \text{ and } e'' \notin E_{\text{redundant}}$

However, this contradicts how we chose e . □

With this lemma in hand, we can now prove Theorem 5.9

Proof of Theorem 5.9. Let T be a minimum spanning tree of G and assume that there is an edge e such that $e \notin E_{\text{redundant}}$ and $e \notin E(T)$. Since T is a spanning tree of G , $E(T) \cup \{e\}$ contains a cycle C . Since $e \notin E_{\text{redundant}}$, this cycle C must contain an edge e' such that $l(e') > l(e)$. Now observe that $T' = (T \setminus \{e'\}) \cup \{e\}$ is also a spanning tree and has smaller total length than T , which contradicts the assumption that T is a minimum spanning tree.

Conversely, if $e \in E_{\text{redundant}}$ then by Lemma 5.10 there is a cycle C such that e is the longest edge in C and all other edges $e' \in E(C)$ are not in $E_{\text{redundant}}$. By the above argument, $E(C) \setminus \{e\} \subseteq E(T)$ which implies that $e' \notin E(T)$ as otherwise T would contain a cycle. $\square$

We now use Theorem 5.9 to show that Kruskal's algorithm and Prim's algorithm both give the minimum spanning tree.

To show that Kruskal's algorithm gives the minimum spanning tree T , we need to show that it rejects every edge in $E_{\text{redundant}}$ and accepts every edge in $E(G) \setminus E_{\text{redundant}}$. We can prove this by induction.

The base case is trivial as Kruskal's algorithm will always accept the edge which has the smallest length. For the inductive step, assume that Kruskal's algorithm is correct on the edges so far and consider the next edge e . There are two cases to consider:

1. If $e \notin E_{\text{redundant}}$ then $T_{\text{current}} \cup \{e\}$ does not contain a cycle because $T_{\text{current}} \cup \{e\} \subseteq T$ and T contains no cycles. Thus, Kruskal's algorithm will correctly accept e .
2. If $e \in E_{\text{redundant}}$ then by Lemma 5.10, there exists a cycle C containing e such that every other edge of C is not in $E_{\text{redundant}}$ and has smaller length than e . By the inductive hypothesis, all of these edges will be in T_{current} so $T_{\text{current}} \cup \{e\}$ contains C and thus Kruskal's algorithm will correctly reject e .

To show that Prim's algorithm gives the minimum spanning tree, observe that Prim's algorithm only accepts an edge e if e is the shortest edge between S and $V \setminus S$ where S is the set of vertices which have been reached so far. Such an edge e can never be in $E_{\text{redundant}}$ because any cycle containing e must contain another edge between S and $V \setminus S$ and any such edge is longer than e . Thus, Prim's algorithm never accepts an edge in $E_{\text{redundant}}$.

To see that Prim's algorithm accepts all edges which are not in $E_{\text{redundant}}$, just as with Kruskal's algorithm, observe that at the time an edge $e \notin E_{\text{redundant}}$ is considered, $T_{\text{current}} \subseteq T$ and thus $T_{\text{current}} \cup \{e\} \subseteq T$ so e does not form a cycle. Thus, e must go to a vertex $w \notin S$ so e is accepted.